Assignment 6

Implementation of Data Integrity Constraints

PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK and DEFAULT constraints, with SQL tests that prove they work

Project: Library Management System **Name:** _____ **Roll No:** _____

1. Objective

Data integrity means the data in the database is **accurate, consistent and valid** at all times. The database engine enforces integrity through *constraints*: rules attached to tables that the DBMS checks on every INSERT, UPDATE and DELETE. In Assignment 5 the tables were created with primary keys, basic foreign keys and a few UNIQUE/NOT NULL rules. This assignment strengthens the design with **named foreign keys with referential actions, additional UNIQUE constraints, CHECK constraints and DEFAULT values**, explains the purpose of each, and then **executes SQL statements that try to break the rules** to prove that the integrity of the data and the relationships between tables are preserved.

Integrity type	Constraint used	What it protects
Entity integrity	PRIMARY KEY, NOT NULL	Every row is uniquely identifiable and has its mandatory values
Referential integrity	FOREIGN KEY (with ON UPDATE / ON DELETE actions)	Relationships between tables never point to missing rows
Domain integrity	CHECK, DEFAULT, data types	Column values come from a valid set / range, and sensible values are filled in automatically
Uniqueness integrity	UNIQUE	No duplicate ISBNs, e-mails, duplicate transactions, etc.

2. Starting Point and Gaps in the Assignment 5 Design

The Assignment 5 script already defines primary keys, foreign keys (auto-named, with the default RESTRICT behaviour), UNIQUE on *ISBN*, *Email* and *Fines.LoanID*, and NOT NULL on key attributes. Because the tables allow many other invalid combinations, the following gaps are closed in this assignment:

- A loan, reservation or fine could be saved with a **NULL foreign key** (a loan with no member).
- Nothing stops **AvailableCopies > TotalCopies**, negative copy counts, or a **DueDate earlier than the IssueDate**.
- Status columns (*Loans.Status, Fines.PaidStatus, Members.MembershipType*) accept **any text**, including typos.
- A fine can be marked "Paid" with **no payment date**; a loan can be "Returned" with **no return date**.
- Duplicate publishers, categories, staff e-mails, or the same transaction recorded twice are allowed.
- The deletion/update behaviour of foreign keys was never chosen deliberately.

3. Implementing the Constraints

All statements below are in the script `06_constraints.sql` and were executed against the database created in Assignment 5 (the tables are still empty, so no existing data can conflict with a new rule). Each part is shown with a table explaining the purpose of every constraint.

3.1 Primary keys

Every table already has a single-column INT PRIMARY KEY (defined in Assignment 5). A primary key is automatically **NOT NULL and UNIQUE**, and the engine builds an index on it, so every row is identifiable and lookups/joins are fast.

Primary keys currently defined

```
SELECT TABLE_NAME AS table_name, COLUMN_NAME AS primary_key_column
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'LibraryManagementSystem' AND CONSTRAINT_NAME = 'PRIMARY'
ORDER BY TABLE_NAME;
```

Output: (9 rows)

table_name	primary_key_column
Authors	AuthorID
Books	BookID
Categories	CategoryID
Fines	FineID
Loans	LoanID
Members	MemberID
Publishers	PublisherID
Reservations	ReservationID
Staff	StaffID

3.2 Foreign keys

Part A - mandatory relationships and named foreign keys with referential actions

```
-- (Primary keys already exist from Assignment 5.)

-- A1. Mandatory relationships: child rows must always point to a parent
ALTER TABLE Loans
    MODIFY BookID   INT NOT NULL,
    MODIFY MemberID INT NOT NULL,
    MODIFY StaffID  INT NOT NULL;

ALTER TABLE Reservations
    MODIFY BookID   INT NOT NULL,
    MODIFY MemberID INT NOT NULL;

ALTER TABLE Fines
    MODIFY LoanID INT NOT NULL;

-- A2. Replace the auto-named foreign keys with named ones that have
--     explicit referential actions
ALTER TABLE Books
    DROP FOREIGN KEY Books_ibfk_1,
    DROP FOREIGN KEY Books_ibfk_2,
    DROP FOREIGN KEY Books_ibfk_3;

ALTER TABLE Books
    ADD CONSTRAINT fk_books_author FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    ADD CONSTRAINT fk_books_publisher FOREIGN KEY (PublisherID) REFERENCES Publishers(PublisherID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    ADD CONSTRAINT fk_books_category FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
        ON UPDATE CASCADE ON DELETE RESTRICT;

ALTER TABLE Loans
    DROP FOREIGN KEY Loans_ibfk_1,
    DROP FOREIGN KEY Loans_ibfk_2,
    DROP FOREIGN KEY Loans_ibfk_3;

ALTER TABLE Loans
    ADD CONSTRAINT fk_loans_book FOREIGN KEY (BookID) REFERENCES Books(BookID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    ADD CONSTRAINT fk_loans_member FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    ADD CONSTRAINT fk_loans_staff FOREIGN KEY (StaffID) REFERENCES Staff(StaffID)
        ON UPDATE CASCADE ON DELETE RESTRICT;

ALTER TABLE Reservations
    DROP FOREIGN KEY Reservations_ibfk_1,
    DROP FOREIGN KEY Reservations_ibfk_2;

ALTER TABLE Reservations
    ADD CONSTRAINT fk_res_book FOREIGN KEY (BookID) REFERENCES Books(BookID)
        ON UPDATE CASCADE ON DELETE CASCADE,
    ADD CONSTRAINT fk_res_member FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
        ON UPDATE CASCADE ON DELETE CASCADE;

ALTER TABLE Fines
    DROP FOREIGN KEY Fines_ibfk_1;

ALTER TABLE Fines
    ADD CONSTRAINT fk_fines_loan FOREIGN KEY (LoanID) REFERENCES Loans(LoanID)
        ON UPDATE CASCADE ON DELETE RESTRICT;
```

11 statements executed successfully (Query OK, 0 rows affected each).

Foreign key	References	ON UPDATE / ON DELETE	Purpose and reason for the chosen action
fk_books_author	Authors(AuthorID)	CASCADE / RESTRICT	A book must belong to a real author. An author who still has books cannot be deleted (protects the catalogue); a changed AuthorID is propagated.
fk_books_publisher	Publishers(PublisherID)	CASCADE / RESTRICT	Same rule for publishers.
fk_books_category	Categories(CategoryID)	CASCADE / RESTRICT	Same rule for categories.
fk_loans_book	Books(BookID)	CASCADE / RESTRICT	A loan must refer to an existing book; a book with borrowing history cannot be deleted (history must be kept).

Foreign key	References	ON UPDATE / ON DELETE	Purpose and reason for the chosen action
fk_loans_member	Members(MemberID)	CASCADE / RESTRICT	A loan must refer to an existing member; members with loan history cannot be erased.
fk_loans_staff	Staff(StaffID)	CASCADE / RESTRICT	Every issue is traceable to the staff member who handled it.
fk_res_book	Books(BookID)	CASCADE / CASCADE	A reservation has no meaning without its book or member, so reservations are removed with them.
fk_res_member	Members(MemberID)	CASCADE / CASCADE	Same as above.
fk_fines_loan	Loans(LoanID)	CASCADE / RESTRICT	A fine must belong to a real loan; financial records must never disappear silently when a loan is deleted.

NOT NULL on foreign-key columns (Loans.BookID/MemberID/StaffID, Reservations.BookID/MemberID, Fines.LoanID): a foreign key may legally be NULL, so NOT NULL is needed to make the relationship *mandatory*.

3.3 UNIQUE constraints

Part B - UNIQUE constraints

```
-- (Books.ISBN, Members.Email and Fines.LoanID are already UNIQUE.)
ALTER TABLE Publishers   ADD CONSTRAINT uq_publisher_name   UNIQUE (PublisherName);
ALTER TABLE Categories   ADD CONSTRAINT uq_category_name    UNIQUE (CategoryName);
ALTER TABLE Staff        ADD CONSTRAINT uq_staff_email      UNIQUE (Email);
ALTER TABLE Authors      ADD CONSTRAINT uq_author_identity  UNIQUE (AuthorName, BirthYear);
ALTER TABLE Loans        ADD CONSTRAINT uq_loan_event       UNIQUE (BookID, MemberID, IssueDate);
ALTER TABLE Reservations ADD CONSTRAINT uq_reservation_event UNIQUE (BookID, MemberID, ReservationDate);
```

6 statements executed successfully (Query OK, 0 rows affected each).

Constraint	Table (columns)	Purpose
(existing) ISBN	Books (ISBN)	Each ISBN identifies one catalogue entry; prevents duplicate book records.
(existing) Email	Members (Email)	One account per e-mail address.
(existing) LoanID	Fines (LoanID)	A loan carries at most one fine record (one-to-one relationship).
uq_publisher_name	Publishers (PublisherName)	The same publisher cannot be entered twice.
uq_category_name	Categories (CategoryName)	No duplicate category names.
uq_staff_email	Staff (Email)	Each staff member has a distinct e-mail address.
uq_author_identity	Authors (AuthorName, BirthYear)	Prevents the same author being stored twice (composite key; two different authors with the same name are still allowed if their birth years differ).
uq_loan_event	Loans (BookID, MemberID, IssueDate)	The same book cannot be issued to the same member twice on the same day (duplicate transaction).
uq_reservation_event	Reservations (BookID, MemberID, ReservationDate)	A member cannot reserve the same book twice on the same day.

3.4 DEFAULT constraints

Part C - DEFAULT values (together with NOT NULL)

```
ALTER TABLE Books
    MODIFY TotalCopies     INT NOT NULL DEFAULT 1,
    MODIFY AvailableCopies INT NOT NULL DEFAULT 1;

ALTER TABLE Members
    MODIFY MembershipDate DATE        NOT NULL DEFAULT (CURDATE()),
    MODIFY MembershipType VARCHAR(20) NOT NULL DEFAULT 'Standard';

ALTER TABLE Staff
    MODIFY Role     VARCHAR(50) NOT NULL DEFAULT 'Library Clerk',
    MODIFY HireDate DATE        NOT NULL DEFAULT (CURDATE());

ALTER TABLE Loans
    MODIFY IssueDate DATE        NOT NULL DEFAULT (CURDATE()),
    MODIFY DueDate   DATE        NOT NULL DEFAULT (CURDATE() + INTERVAL 14 DAY),
    MODIFY Status    VARCHAR(20) NOT NULL DEFAULT 'Issued';

ALTER TABLE Reservations
    MODIFY ReservationDate DATE        NOT NULL DEFAULT (CURDATE()),
    MODIFY Status          VARCHAR(20) NOT NULL DEFAULT 'Pending';
ALTER TABLE Fines
    MODIFY PaidStatus VARCHAR(10) NOT NULL DEFAULT 'Unpaid';
```

6 statements executed successfully (Query OK, 0 rows affected each).

Column	Default value	Purpose
Books.TotalCopies / AvailableCopies	1	A newly catalogued book has one copy unless stated otherwise.
Members.MembershipDate	CURDATE()	Registration date is the day of insertion.
Members.MembershipType	'Standard'	New members start on the standard plan.
Staff.Role / HireDate	'Library Clerk' / CURDATE()	Most common role; hire date is the insertion day.
Loans.IssueDate	CURDATE()	A book is issued today unless a date is given.
Loans.DueDate	CURDATE() + 14 days	Implements the library rule of a 14-day loan period.
Loans.Status	'Issued'	A new loan is by definition an issued (open) loan.
Reservations.ReservationDate / Status	CURDATE() / 'Pending'	A new reservation is dated today and waits to be fulfilled.
Fines.PaidStatus	'Unpaid'	A fine is unpaid when it is first raised.

3.5 CHECK constraints

Part D - CHECK constraints (domain integrity)

```
ALTER TABLE Authors
    ADD CONSTRAINT chk_author_birthyear CHECK (BirthYear IS NULL OR BirthYear BETWEEN 1000 AND 2025);

ALTER TABLE Publishers
    ADD CONSTRAINT chk_publisher_phone CHECK (Phone IS NULL OR Phone REGEXP '^[0-9+][0-9 -]{6,14}$');

ALTER TABLE Books
    ADD CONSTRAINT chk_books_copies CHECK (
        TotalCopies >= 0 AND AvailableCopies >= 0 AND AvailableCopies <= TotalCopies),
    ADD CONSTRAINT chk_books_year   CHECK (PublicationYear IS NULL OR PublicationYear BETWEEN 1450 AND 2100),
    ADD CONSTRAINT chk_books_isbn   CHECK (ISBN IS NULL OR ISBN REGEXP '^[0-9X-]{10,17}$');

ALTER TABLE Members
    ADD CONSTRAINT chk_member_type  CHECK (MembershipType IN ('Standard', 'Student', 'Premium', 'Senior')),
    ADD CONSTRAINT chk_member_email CHECK (Email IS NULL OR Email LIKE '%_@_%._%'),
    ADD CONSTRAINT chk_member_phone CHECK (Phone IS NULL OR Phone REGEXP '^[0-9+][0-9 -]{6,14}$');

ALTER TABLE Staff
    ADD CONSTRAINT chk_staff_role  CHECK (Role IN ('Head Librarian', 'Assistant Librarian', 'Library Clerk',
        'Library Assistant', 'Cataloguer')),
    ADD CONSTRAINT chk_staff_phone CHECK (Phone IS NULL OR Phone REGEXP '^[0-9+][0-9 -]{6,14}$');

ALTER TABLE Loans
    ADD CONSTRAINT chk_loan_due    CHECK (DueDate >= IssueDate),
    ADD CONSTRAINT chk_loan_return CHECK (ReturnDate IS NULL OR ReturnDate >= IssueDate),
    ADD CONSTRAINT chk_loan_status CHECK (Status IN ('Issued', 'Returned', 'Overdue')),
    ADD CONSTRAINT chk_loan_status_return CHECK (
        (Status = 'Returned' AND ReturnDate IS NOT NULL) OR
        (Status IN ('Issued', 'Overdue') AND ReturnDate IS NULL));

ALTER TABLE Reservations
    ADD CONSTRAINT chk_res_status CHECK (Status IN ('Pending', 'Fulfilled', 'Cancelled'));

ALTER TABLE Fines
    ADD CONSTRAINT chk_fine_amount CHECK (Amount > 0),
    ADD CONSTRAINT chk_fine_status CHECK (PaidStatus IN ('Paid', 'Unpaid')),
    ADD CONSTRAINT chk_fine_paid_consistency CHECK (
        (PaidStatus = 'Paid'   AND PaidDate IS NOT NULL) OR
        (PaidStatus = 'Unpaid' AND PaidDate IS NULL));
```

8 statements executed successfully (Query OK, 0 rows affected each).

Constraint	Table	Rule	Purpose
chk_author_birthyear	Authors	BirthYear between 1000 and 2025 (or NULL)	Rejects impossible years.
chk_publisher_phone / chk_member_phone / chk_staff_phone	Publishers, Members, Staff	Phone matches digits, "+", space, "-" (7-15 chars)	Rejects letters and garbage in phone numbers.
chk_books_copies	Books	TotalCopies >= 0, AvailableCopies >= 0, AvailableCopies <= TotalCopies	Stock can never be negative or exceed the owned copies.
chk_books_year	Books	PublicationYear between 1450 and 2100	Valid publication years only (printing press onwards).
chk_books_isbn	Books	ISBN has 10-17 characters from 0-9, X, "-"	Basic ISBN format check.
chk_member_type	Members	Standard, Student, Premium or Senior	Stops typos and unknown plans.
chk_member_email	Members	contains '@' and a dot after it	Basic e-mail format validation.
chk_staff_role	Staff	one of five defined roles	Controlled vocabulary for roles.
chk_loan_due	Loans	DueDate >= IssueDate	A loan cannot be due before it was issued.
chk_loan_return	Loans	ReturnDate is NULL or >= IssueDate	A book cannot be returned before it was issued.
chk_loan_status	Loans	Issued, Returned or Overdue	Valid statuses only.
chk_loan_status_return	Loans	Returned <=> ReturnDate present; Issued/Overdue <=> ReturnDate absent	Status and ReturnDate can never contradict each other.
chk_res_status	Reservations	Pending, Fulfilled or Cancelled	Valid statuses only.
chk_fine_amount	Fines	Amount > 0	A fine must be a positive amount.
chk_fine_status	Fines	Paid or Unpaid	Valid statuses only.
chk_fine_paid_consistency	Fines	Paid <=> PaidDate present; Unpaid <=> PaidDate absent	A fine cannot be "Paid" without a payment date.

Note on CHECK: CHECK constraints are enforced from MySQL 8.0.16 and MariaDB 10.2.1 onwards. Older MySQL versions parse CHECK but silently ignore it. Rules that depend on the current date (for example "HireDate must not be in the future") cannot be written as CHECK constraints because CHECK expressions must be deterministic.

4. Verifying that the Constraints Exist

Number of constraints of each type per table

```
SELECT TABLE_NAME AS table_name,
       SUM(CONSTRAINT_TYPE = 'PRIMARY KEY') AS pk,
       SUM(CONSTRAINT_TYPE = 'FOREIGN KEY') AS fk,
       SUM(CONSTRAINT_TYPE = 'UNIQUE')      AS uq,
       SUM(CONSTRAINT_TYPE = 'CHECK')       AS chk
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = 'LibraryManagementSystem'
GROUP BY TABLE_NAME
ORDER BY TABLE_NAME;
```

Output: (9 rows)

table_name	pk	fk	uq	chk
Authors	1	0	1	1
Books	1	3	1	3
Categories	1	0	1	0
Fines	1	1	1	3
Loans	1	3	1	4
Members	1	0	1	3
Publishers	1	0	1	1
Reservations	1	2	1	1
Staff	1	0	1	2

Explanation: Totals: 9 primary keys, 9 foreign keys, 9 unique constraints (3 original + 6 new) and 18 CHECK constraints.

Foreign keys with their referential actions

```
SELECT rc.CONSTRAINT_NAME AS fk_name, rc.TABLE_NAME AS child_table,
       rc.REFERENCED_TABLE_NAME AS parent_table, rc.UPDATE_RULE, rc.DELETE_RULE
FROM information_schema.REFERENTIAL_CONSTRAINTS rc
WHERE rc.CONSTRAINT_SCHEMA = 'LibraryManagementSystem'
ORDER BY rc.TABLE_NAME, rc.CONSTRAINT_NAME;
```

Output: (9 rows)

fk_name	child_table	parent_table	UPDATE_RULE	DELETE_RULE
fk_books_author	Books	Authors	CASCADE	RESTRICT
fk_books_category	Books	Categories	CASCADE	RESTRICT
fk_books_publisher	Books	Publishers	CASCADE	RESTRICT
fk_fines_loan	Fines	Loans	CASCADE	RESTRICT
fk_loans_book	Loans	Books	CASCADE	RESTRICT
fk_loans_member	Loans	Members	CASCADE	RESTRICT
fk_loans_staff	Loans	Staff	CASCADE	RESTRICT
fk_res_book	Reservations	Books	CASCADE	CASCADE
fk_res_member	Reservations	Members	CASCADE	CASCADE

Columns that now have DEFAULT values

```
SELECT TABLE_NAME AS table_name, COLUMN_NAME AS column_name, COLUMN_DEFAULT AS default_value
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = 'LibraryManagementSystem' AND COLUMN_DEFAULT IS NOT NULL AND COLUMN_DEFAULT <> 'NULL'
ORDER BY TABLE_NAME, ORDINAL_POSITION;
```

Output: (12 rows)

table_name	column_name	default_value
Books	TotalCopies	1
Books	AvailableCopies	1
Fines	PaidStatus	'Unpaid'
Loans	IssueDate	curdate()
Loans	DueDate	(curdate() + interval 14 day)
Loans	Status	'Issued'
Members	MembershipDate	curdate()
Members	MembershipType	'Standard'
Reservations	ReservationDate	curdate()
Reservations	Status	'Pending'
Staff	Role	'Library Clerk'
Staff	HireDate	curdate()

5. Testing the Constraints with SQL Statements

The statements below deliberately try to violate each rule. Every test is run inside a transaction that is **rolled back** at the end, so the tables are left empty and ready for the sample data of Assignment 7. A red box is the error message returned by the database server, which is exactly the result we want: the invalid change is rejected and nothing is stored. (Error codes shown are from MariaDB; on MySQL 8 a CHECK violation is error 3819 instead of 4025. The CURDATE() values in the DEFAULT tests come from the server clock, here 26 October 2026.)

Test setup: insert one valid row in each parent table (inside a transaction)

```
START TRANSACTION;

INSERT INTO Authors    VALUES (901, 'Test Author', 'Indian', 1970);
INSERT INTO Publishers VALUES (901, 'Test Publisher', 'Bengaluru', '080-12345678');
INSERT INTO Categories VALUES (901, 'Test Category', 'Temporary test data');
INSERT INTO Books (BookID, Title, ISBN, AuthorID, PublisherID, CategoryID, PublicationYear, TotalCopies,
    AvailableCopies)
VALUES (901, 'Test Book', '978-1-111-11111-1', 901, 901, 901, 2020, 3, 3);
INSERT INTO Members (MemberID, FullName, Email) VALUES (901, 'Test Member', 'test.member@example.com');
INSERT INTO Staff (StaffID, FullName) VALUES (901, 'Test Staff');
```

Explanation: These valid rows are accepted. Note that Members and Staff were inserted with only a few columns: the DEFAULT constraints filled in the rest (tested in 5.5).

Setup: Query OK - six valid rows inserted (still inside the open transaction).

5.1 PRIMARY KEY - duplicate and NULL keys are rejected

Test 1: insert an author with an AuthorID that already exists

```
INSERT INTO Authors VALUES (901, 'Another Author', 'Indian', 1980);
```

Output:

```
ERROR 1062: Duplicate entry '901' for key 'PRIMARY'
```

Explanation: Primary key values must be unique, so the duplicate AuthorID 901 is rejected.

Test 2: insert an author with a NULL primary key

```
INSERT INTO Authors (AuthorID, AuthorName) VALUES (NULL, 'No Key Author');
```

Output:

```
ERROR 1048: Column 'AuthorID' cannot be null
```

Explanation: A primary key column is implicitly NOT NULL.

5.2 FOREIGN KEY - referential integrity

Test 3: insert a book that refers to a non-existent author (AuthorID 999)

```
INSERT INTO Books (BookID, Title, ISBN, AuthorID, PublisherID, CategoryID, TotalCopies, AvailableCopies)
VALUES (902, 'Orphan Book', '978-2-222-22222-2', 999, 901, 901, 1, 1);
```

Output:

```
ERROR 1452: Cannot add or update a child row: a foreign key constraint fails (`LibraryManagementSystem`.`Books`,
CONSTRAINT `fk_books_author` FOREIGN KEY (`AuthorID`) REFERENCES `Authors` (`AuthorID`) ON UPDATE CASCADE)
```

Explanation: The child row would point to a missing parent, so the insert fails: no "orphan" rows can exist.

Test 4: delete an author who still has a book (ON DELETE RESTRICT)

```
DELETE FROM Authors WHERE AuthorID = 901;
```

Output:

```
ERROR 1451: Cannot delete or update a parent row: a foreign key constraint fails (`LibraryManagementSystem`.`Books`,
CONSTRAINT `fk_books_author` FOREIGN KEY (`AuthorID`) REFERENCES `Authors` (`AuthorID`) ON UPDATE CASCADE)
```

Explanation: Author 901 is referenced by Book 901, so the deletion is blocked; this protects the catalogue.

Test 5: change an author's primary key (ON UPDATE CASCADE)

```
SELECT BookID, AuthorID FROM Books WHERE BookID = 901;
UPDATE Authors SET AuthorID = 902 WHERE AuthorID = 901;
SELECT BookID, AuthorID FROM Books WHERE BookID = 901;
UPDATE Authors SET AuthorID = 901 WHERE AuthorID = 902;
```

Output: [1/2] (1 row)

BookID	AuthorID
901	901

Output: [2/2] (1 row)

BookID	AuthorID
901	902

Explanation: After the author's key changed from 901 to 902, the book's foreign key followed automatically, so the relationship stayed intact. The last statement restores the original value.

Test 6: insert a loan without a member (NOT NULL on the foreign key)

```
INSERT INTO Loans (LoanID, BookID, MemberID, StaffID, IssueDate, DueDate, ReturnDate, Status)
VALUES (902, 901, NULL, 901, '2026-10-20', '2026-11-03', NULL, 'Issued');
```

Output:

```
ERROR 1048: Column 'MemberID' cannot be null
```

Explanation: A loan must always belong to a member, so the NULL foreign key is rejected.

5.3 UNIQUE - duplicates are rejected

Test 7: insert a second book with an ISBN that already exists

```
INSERT INTO Books (BookID, Title, ISBN, AuthorID, PublisherID, CategoryID, TotalCopies, AvailableCopies)
VALUES (903, 'Duplicate ISBN Book', '978-1-111-11111-1', 901, 901, 901, 1, 1);
```

Output:

```
ERROR 1062: Duplicate entry '978-1-111-11111-1' for key 'ISBN'
```

Test 8: register a second member with an existing e-mail address

```
INSERT INTO Members (MemberID, FullName, Email) VALUES (904, 'Another Member', 'test.member@example.com');
```

Output:

```
ERROR 1062: Duplicate entry 'test.member@example.com' for key 'Email'
```

Test 9: record the same transaction twice (same book, member and issue date)

```
INSERT INTO Loans (LoanID, BookID, MemberID, StaffID, IssueDate, DueDate, Status)
VALUES (901, 901, 901, 901, '2026-10-20', '2026-11-03', 'Issued');
INSERT INTO Loans (LoanID, BookID, MemberID, StaffID, IssueDate, DueDate, Status)
VALUES (903, 901, 901, 901, '2026-10-20', '2026-11-03', 'Issued');
```

Output:

```
ERROR 1062: Duplicate entry '901-901-2026-10-20' for key 'uq_loan_event'
```

Explanation: The first loan is accepted; the identical second loan violates uq_loan_event and is rejected.

5.4 CHECK - invalid values are rejected

Test 10: more available copies (5) than total copies (2)

```
INSERT INTO Books (BookID, Title, ISBN, AuthorID, PublisherID, CategoryID, TotalCopies, AvailableCopies)
VALUES (904, 'Impossible Stock', '978-4-444-44444-4', 901, 901, 901, 2, 5);
```

Output:

```
ERROR 4025: CONSTRAINT `chk_books_copies` failed for `LibraryManagementSystem`.`Books`
```

Test 11: a loan whose due date is before its issue date

```
INSERT INTO Loans (LoanID, BookID, MemberID, StaffID, IssueDate, DueDate, Status)
VALUES (904, 901, 901, 901, '2026-10-25', '2026-10-10', 'Issued');
```

Output:

```
ERROR 4025: CONSTRAINT `chk_loan_due` failed for `LibraryManagementSystem`.`Loans`
```

Test 12: a loan marked "Returned" but with no return date

```
INSERT INTO Loans (LoanID, BookID, MemberID, StaffID, IssueDate, DueDate, ReturnDate, Status)
VALUES (905, 901, 901, 901, '2026-10-01', '2026-10-15', NULL, 'Returned');
```

Output:

```
ERROR 4025: CONSTRAINT `chk_loan_status_return` failed for `LibraryManagementSystem`.`Loans`
```

Test 13: a membership type that is not in the allowed list

```
INSERT INTO Members (MemberID, FullName, Email, MembershipType)
VALUES (905, 'Gold Member', 'gold.member@example.com', 'Gold');
```

Output:

```
ERROR 4025: CONSTRAINT `chk_member_type` failed for `LibraryManagementSystem`.`Members`
```

Test 14: an e-mail address without "@"

```
INSERT INTO Members (MemberID, FullName, Email) VALUES (906, 'Bad Email', 'not-an-email');
```

Output:

```
ERROR 4025: CONSTRAINT `chk_member_email` failed for `LibraryManagementSystem`.`Members`
```

Test 15: a fine marked "Paid" with no payment date

```
INSERT INTO Fines (FineID, LoanID, Amount, PaidStatus, PaidDate) VALUES (901, 901, 14.00, 'Paid', NULL);
```

Output:

```
ERROR 4025: CONSTRAINT `chk_fine_paid_consistency` failed for `LibraryManagementSystem`.`Fines`
```

Test 16: a fine with a negative amount

```
INSERT INTO Fines (FineID, LoanID, Amount) VALUES (902, 901, -5.00);
```

Output:

```
ERROR 4025: CONSTRAINT `chk_fine_amount` failed for `LibraryManagementSystem`.`Fines`
```

5.5 DEFAULT - missing values are filled in automatically

Test 17: the Members and Staff rows inserted in the setup supplied only an ID, a name and (for Members) an e-mail

```
SELECT MemberID, FullName, MembershipDate, MembershipType FROM Members WHERE MemberID = 901;
SELECT StaffID, FullName, Role, HireDate FROM Staff WHERE StaffID = 901;
```

Output: [1/2] (1 row)

MemberID	FullName	MembershipDate	MembershipType
901	Test Member	2026-10-26	Standard

Output: [2/2] (1 row)

StaffID	FullName	Role	HireDate
901	Test Staff	Library Clerk	2026-10-26

Explanation: MembershipDate and HireDate defaulted to today, MembershipType to Standard and Role to Library Clerk.

Test 18: insert a loan, a fine and a reservation giving only the essential columns

```
INSERT INTO Loans (LoanID, BookID, MemberID, StaffID) VALUES (906, 901, 901, 901);
INSERT INTO Fines (FineID, LoanID, Amount) VALUES (903, 906, 14.00);
INSERT INTO Reservations (ReservationID, BookID, MemberID) VALUES (901, 901, 901);
SELECT LoanID, IssueDate, DueDate, ReturnDate, Status FROM Loans WHERE LoanID = 906;
SELECT FineID, Amount, PaidStatus, PaidDate FROM Fines WHERE FineID = 903;
SELECT ReservationID, ReservationDate, Status FROM Reservations WHERE ReservationID = 901;
```

Output: [1/3] (1 row)

LoanID	IssueDate	DueDate	ReturnDate	Status
906	2026-10-26	2026-11-09	NULL	Issued

Output: [2/3] (1 row)

FineID	Amount	PaidStatus	PaidDate
903	14.00	Unpaid	NULL

Output: [3/3] (1 row)

ReservationID	ReservationDate	Status
901	2026-10-26	Pending

Explanation: The loan was issued today and is due 14 days later with status Issued; the fine is Unpaid; the reservation is dated today and Pending.

5.6 ON DELETE CASCADE - dependent rows are removed automatically

Test 19: delete a member who has only a reservation

```
INSERT INTO Members (MemberID, FullName, Email) VALUES (907, 'Reservation Only Member', 'res.only@example.com');
INSERT INTO Reservations (ReservationID, BookID, MemberID) VALUES (902, 901, 907);
SELECT ReservationID, MemberID FROM Reservations WHERE MemberID = 907;
DELETE FROM Members WHERE MemberID = 907;
SELECT COUNT(*) AS reservations_left FROM Reservations WHERE MemberID = 907;
```

Output: [1/2] (1 row)

ReservationID	MemberID
902	907

Output: [2/2] (1 row)

reservations_left
0

Explanation: Deleting the member also deleted the reservation (ON DELETE CASCADE), so no reservation is left pointing at a missing member.

Test 20: delete a member who has a loan (ON DELETE RESTRICT on Loans)

```
DELETE FROM Members WHERE MemberID = 901;
```

Output:

```
ERROR 1451: Cannot delete or update a parent row: a foreign key constraint fails (`LibraryManagementSystem`.`Loans`,
CONSTRAINT `fk_loans_member` FOREIGN KEY (`MemberID`) REFERENCES `Members` (`MemberID`) ON UPDATE CASCADE)
```

Explanation: Loan history must be preserved, so the member cannot be deleted while loans exist.

5.7 Rolling back the tests

ROLLBACK and confirm that every table is empty again

```
ROLLBACK;

SELECT 'Authors' AS table_name, COUNT(*) AS rows_in_table FROM Authors
UNION ALL SELECT 'Publishers', COUNT(*) FROM Publishers
UNION ALL SELECT 'Categories', COUNT(*) FROM Categories
UNION ALL SELECT 'Books', COUNT(*) FROM Books
UNION ALL SELECT 'Members', COUNT(*) FROM Members
UNION ALL SELECT 'Staff', COUNT(*) FROM Staff
UNION ALL SELECT 'Loans', COUNT(*) FROM Loans
UNION ALL SELECT 'Reservations', COUNT(*) FROM Reservations
UNION ALL SELECT 'Fines', COUNT(*) FROM Fines;
```

Output: (9 rows)

table_name	rows_in_table
Authors	0
Publishers	0
Categories	0
Books	0
Members	0
Staff	0
Loans	0
Reservations	0
Fines	0

Explanation: All test rows were undone; the database is clean for the sample data of Assignment 7.

6. Summary of Test Results

Test	Constraint type	Attempted violation	Result
1-2	PRIMARY KEY	Duplicate and NULL key	Rejected
3	FOREIGN KEY	Book with non-existent author	Rejected
4	FOREIGN KEY (RESTRICT)	Delete an author who has books	Rejected
5	FOREIGN KEY (CASCADE)	Change a parent key	Child updated automatically
6	NOT NULL foreign key	Loan without member	Rejected
7-9	UNIQUE	Duplicate ISBN, e-mail, transaction	Rejected
10-16	CHECK	Bad stock, dates, status, e-mail, fine data	Rejected
17-18	DEFAULT	Omit optional columns	Defaults filled in
19	ON DELETE CASCADE	Delete member with only a reservation	Reservation removed
20	ON DELETE RESTRICT	Delete member with loans	Rejected

Conclusion: with these constraints the database itself refuses invalid data and broken relationships, so correctness no longer depends on every application or user remembering the rules.